Railway Reservation System

EGC 301P – Operating Systems Lab Mini Project

Submitted by

TANMAY DIXIT

BT2024016

Date: April 30, 2026

Contents

1 Problem Statement **2**

2 System Overview **3**

3 Implementation of OS Concepts **5**

3.1 Socket Programming 5

3.2 Role-Based Authorization 6

3.3 File Locking 7

3.4 Concurrency Control 8

3.5 Data Consistency 9

3.6 Inter-Process Communication (IPC) — Pipes 10

4 Admin Operations Demo **12**

5 Challenges Faced and Solutions **13**

6 Conclusion **14**

1. Problem Statement

Concurrent Client-Server Railway Reservation System

Objective: Design and implement a robust, socket-based client-server Railway Reservation System in C that allows multiple users to simultaneously connect, book, and cancel train tickets.

The Challenge: In a multi-user reservation environment, concurrent access to shared data (such as **train schedules** and **ticket inventories**) frequently leads to **race conditions, dirty reads, and lost updates**. The system must successfully handle simultaneous client requests while guaranteeing strict data integrity, preventing unauthorized access, and maintaining an **unblocked audit trail** of system events.

Technical Requirements and Specifications: The proposed solution must be developed utilizing the following core Operating System principles to ensure security, consistency, and efficient resource management:

Socket Programming: Establish reliable, network-based communication between multiple clients and a centralized server over TCP sockets.

Concurrency Control: Manage simultaneous client connections using POSIX threads (pthreads).

Data Consistency: Atomicity for critical operations. Complex sequences, such as the read-check-update-write cycle during ticket booking, must be strictly wrapped within mutex-protected (pthread-mutex) critical sections.

File Locking: Protect shared backend data files from concurrent conflicting modifications by implementing fcntl() file locks, actively preventing data corruption during simultaneous read/write operations.

Role-Based Authorization: Enforce strict access control across three distinct user roles—Admin, Agent, and User.

Inter-Process Communication (IPC): Delegate all system logging responsibilities to a separate child process using fork(), demonstrating asynchronous IPC by transmitting log data from the parent to the child via unnamed pipes (pipe()).

2. System Overview

The system follows a **Client-Server architecture** built over TCP/IP sockets. The server continuously listens on port 9090 and can handle up to five concurrent clients using POSIX threads.

Source Files:

- server.c – main server logic, socket handling, IPC
- client.c – client interface and menu-driven interaction
- users.txt – stores user credentials and roles
- trains.txt – stores train data (number, name, seats)
- bookings.txt – stores all booking records
- server.log – log file written by child process via pipe

User Roles:

- **Admin** – full access (add trains, register users, view all data)
- **Agent** – can book/cancel tickets on behalf of users
- **User** – can view trains, book and cancel own tickets

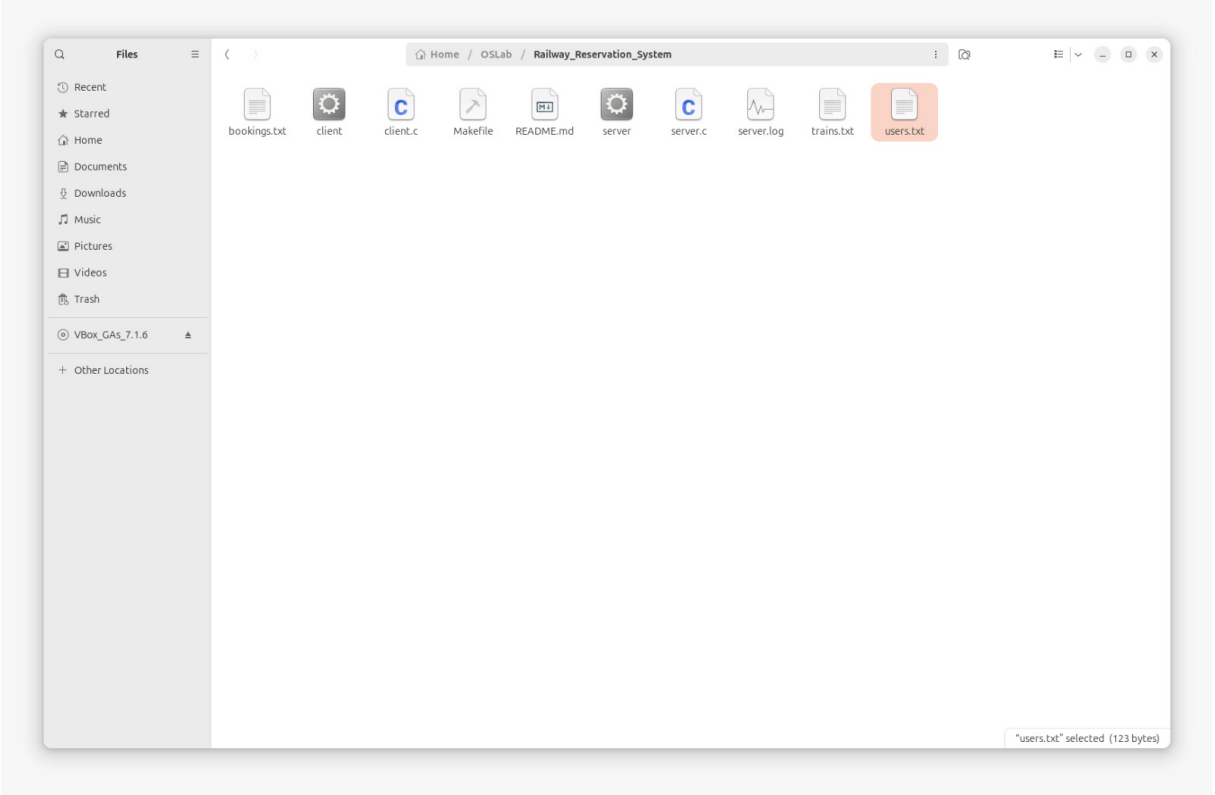

Figure 2.1: Project folder structure in VS Code

3. Implementation of OS Concepts

3.1 Socket Programming

The server creates a TCP socket using socket(AF_INET, SOCK_STREAM, 0), binds it to port 9090 using bind(), and starts listening with listen(). For each incoming connection, accept() returns a new socket descriptor. The client connects via connect() specifying the server's IP and port. Data is exchanged using send() and recv() calls throughout the session.

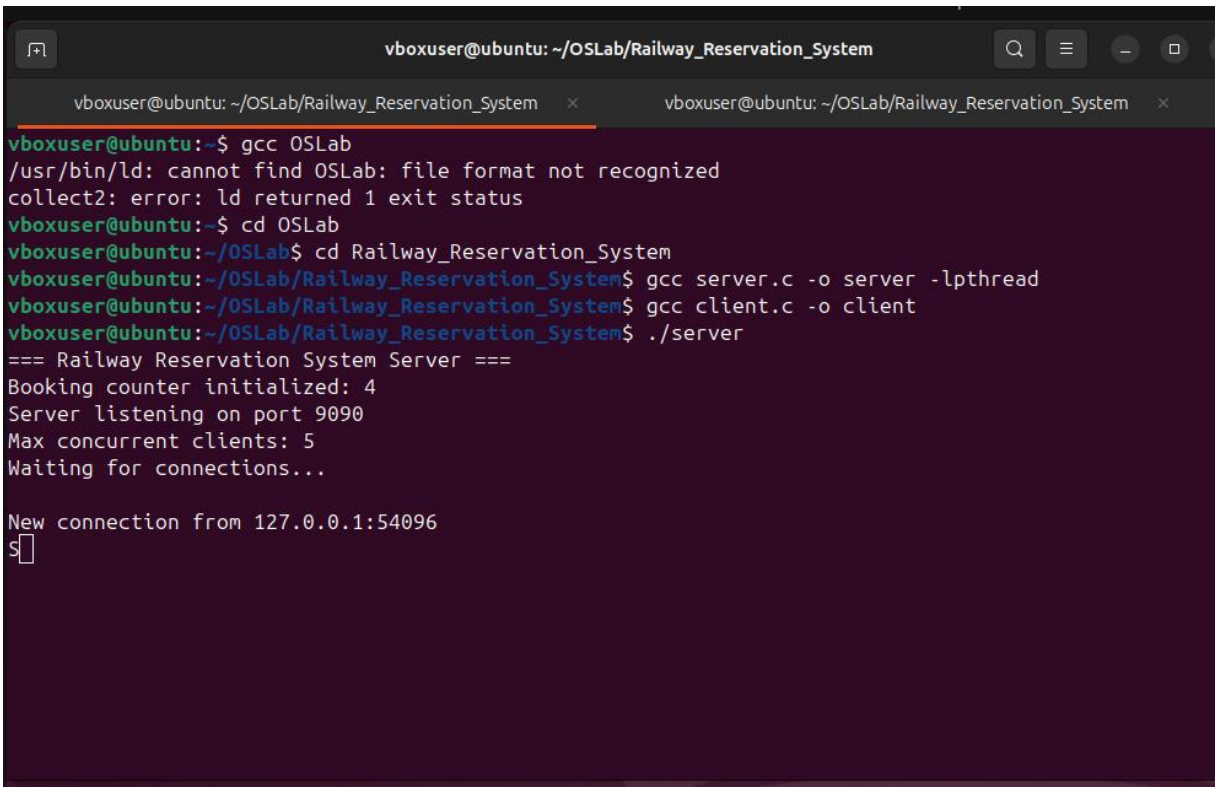

Figure 3.1: Server starting up on port 9090

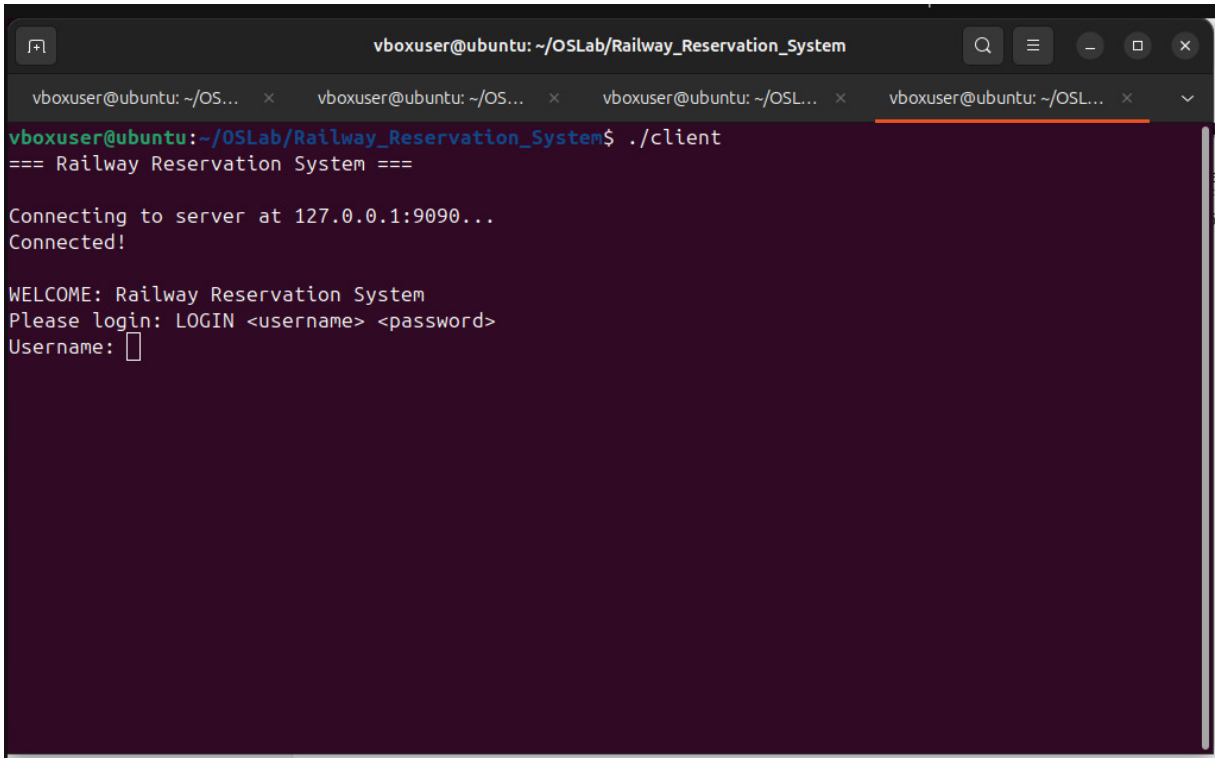

Figure 3.2: Client connecting and displaying login prompt

3.2 Role-Based Authorization

Three roles exist — Admin, Agent, and User. Each role is stored alongside the credentials in users.txt. After a successful login, the server reads the role and presents a menu filtered to the permitted operations. Any attempt to invoke a restricted function is blocked server-side.

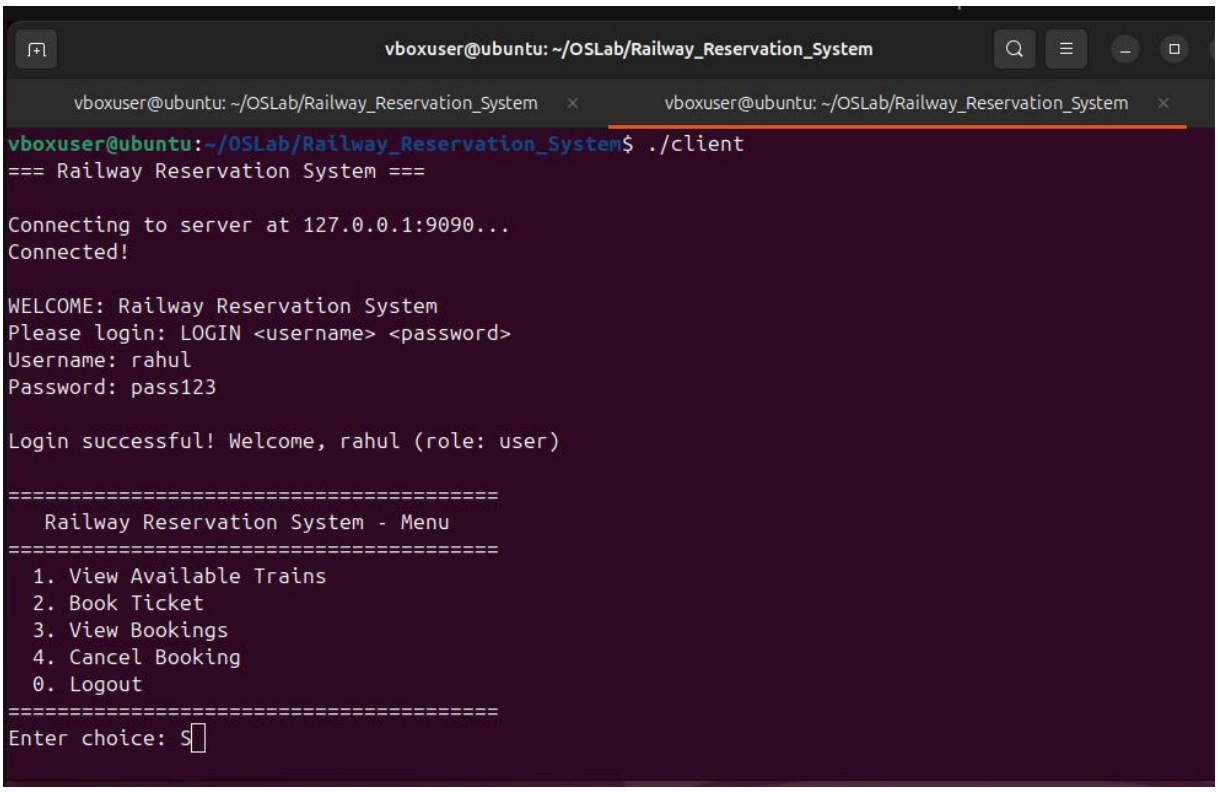

Figure 3.3: User menu after logging in as rahul

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_...
=== Railway Reservation System ===

Connecting to server at 127.0.0.1:9090...
Connected!

WELCOME: Railway Reservation System
Please login: LOGIN <username> <password>
Username: admin
Password: admin123

Login successful! Welcome, admin (role: admin)

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  5. Add New Train (Admin)
  6. Register New User (Admin)
  0. Logout
========================================
Enter choice:
```

Figure 3.4: Admin menu with elevated privileges

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_...
vboxuser@ubuntu:~/OSLab/Railway_Reservation_System$ ./client
=== Railway Reservation System ===

Connecting to server at 127.0.0.1:9090...
Connected!

WELCOME: Railway Reservation System
Please login: LOGIN <username> <password>
Username: rahul
Password: pass123

Login successful! Welcome, rahul (role: user)

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  0. Logout
========================================
Enter choice: 5
Access denied. Admin only.
```

Figure 3.5: Role restriction — admin-only options hidden from User role

3.3 File Locking

The server uses fcntl() with F_RDLCK for shared read locks and F_WRLCK for exclusive write locks. Locks are applied in blocking mode using F_SETLKW, so a thread waits until the lock is available rather than failing immediately. The helper functions lock_file() and unlock_file() encapsulate this logic and are called whenever trains.txt or bookings.txt are read or modified, preventing file corruption under concurrent access.

```c
/* ========================================================
 * FILE LOCKING - uses fcntl for advisory locking
 * This ensures safe access when multiple threads/processes
 * try to read or write the same file.
 * ======================================================== */
void lock_file(int fd, int type) {
    struct flock fl;
    memset(&fl, 0, sizeof(fl));
    fl.l_type   = type;     /* F_RDLCK, F_WRLCK, F_UNLCK */
    fl.l_whence = SEEK_SET;
    fl.l_start  = 0;
    fl.l_len    = 0;        /* 0 means lock entire file */

    if (fcntl(fd, F_SETLKW, &fl) == -1) {
        perror("fcntl lock");
    }
}

void unlock_file(int fd) {
    lock_file(fd, F_UNLCK);
}
```

Figure 3.6: File locking implementation using fcntl()

(Showing error as file is open in windows.)

3.4 Concurrency Control

The server uses two synchronization primitives:

- pthread_mutex_t – protects shared in-memory data (e.g., booking counter) from race conditions between threads.
- sem_t (semaphore, initialized to 5) – limits the number of simultaneously connected clients to prevent resource exhaustion. A client acquires the semaphore slot with sem_wait() on connect and releases it with sem_post() on disconnect.

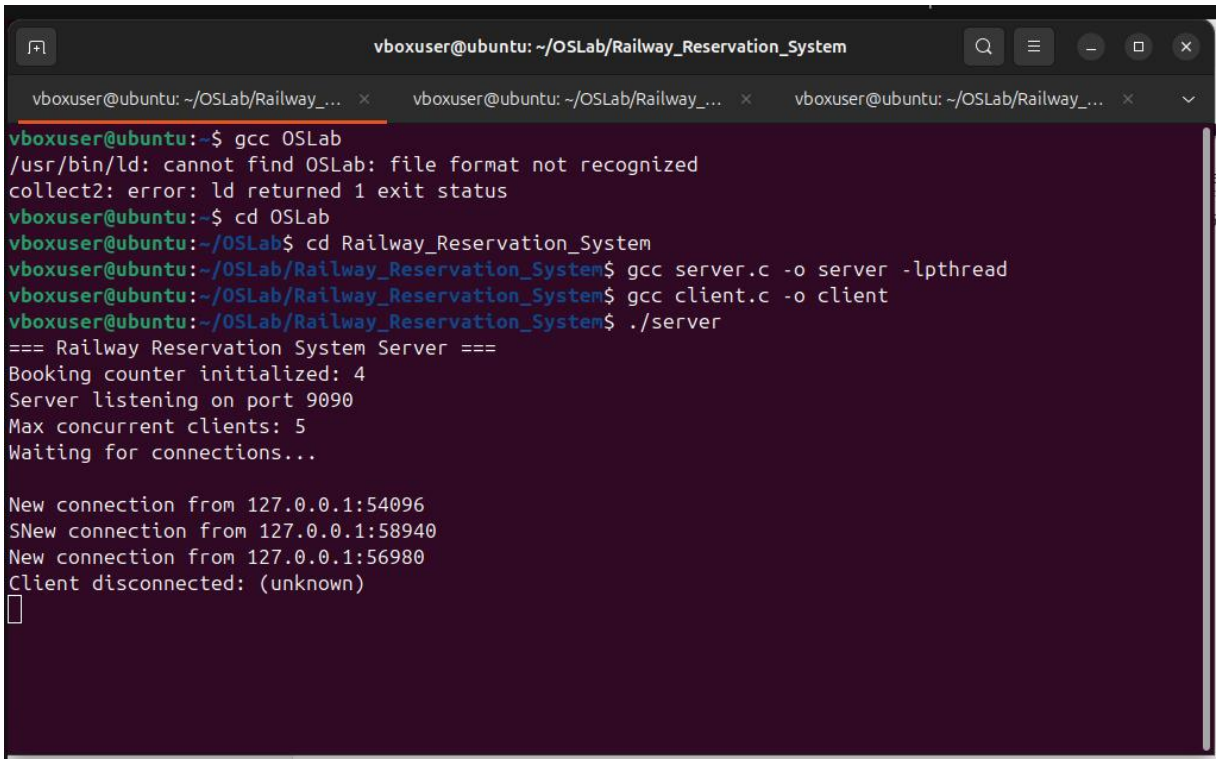

Figure 3.7: Server handling multiple concurrent clients

3.5 Code: Mutex & Semaphore usage

```c
/* ======================================================
 * VIEW TRAINS handler
 * ====================================================== */
void handle_view_trains(int sockfd) {
    /* lock mutex - prevents other threads from modifying file at the same time */
    pthread_mutex_lock(&data_mutex);

    Train trains[MAX_TRAINS];
    int count = read_trains(trains, MAX_TRAINS);

    char response[BUF_SIZE] = "";
    strcat(response, "\n--- Available Trains ---\n");
    strcat(response, "ID   Name                 From         To           Seats  Fare\n");
    strcat(response, "---- ------------------ ------------ ------------ ------ -----\n");

    for (int i = 0; i < count; i++) {
        char row[MAX_LINE];
        snprintf(row, sizeof(row), "%-4d %-18s %-12s %-12s %-6d %d\n",
                 trains[i].id, trains[i].name, trains[i].from,
                 trains[i].to, trains[i].seats, trains[i].fare);
        strcat(response, row);
    }

    if (count == 0)
        strcat(response, "  (no trains found)\n");

    strcat(response, "END\n");
    send_msg(sockfd, response);

    pthread_mutex_unlock(&data_mutex);
}
```

```c
/* ====================================================
 * BOOK TICKET handler
 * Uses mutex + file lock to ensure data consistency.
 * Only one thread can modify seats at a time, preventing
 * race conditions and lost updates.
 * ==================================================== */
void handle_book_ticket(int sockfd, const char *username, int train_id, int num_seats) {
    pthread_mutex_lock(&data_mutex);

    Train trains[MAX_TRAINS];
    int tcount = read_trains(trains, MAX_TRAINS);

    /* find the requested train */
    int idx = -1;
    for (int i = 0; i < tcount; i++) {
        if (trains[i].id == train_id) { idx = i; break; }
    }

    if (idx == -1) {
        send_msg(sockfd, "ERROR: Train not found\nEND\n");
        pthread_mutex_unlock(&data_mutex);
        return;
    }

    if (num_seats <= 0) {
        send_msg(sockfd, "ERROR: Invalid number of seats\nEND\n");
        pthread_mutex_unlock(&data_mutex);
        return;
    }

    if (trains[idx].seats < num_seats) {
        char msg[128];
        snprintf(msg, sizeof(msg), "ERROR: Only %d seats available\nEND\n",
                 trains[idx].seats);
        send_msg(sockfd, msg);
        pthread_mutex_unlock(&data_mutex);
        return;
    }

    /* update seat count */
    trains[idx].seats -= num_seats;
    write_trains(trains, tcount);

    /* create booking record */
    booking_counter++;
    int bid = booking_counter;
    int amount = num_seats * trains[idx].fare;
```

```c
server.c 9+
server.c > main(void)
int main(void) {

    if (listen(server_fd, 10) < 0) {
        perror("listen");
        exit(1);
    }

    printf("Server listening on port %d\n", PORT);
    printf("Max concurrent clients: %d\n", MAX_CLIENTS);
    printf("Waiting for connections...\n\n");

    /* accept loop */
    while (1) {
        struct sockaddr_in client_addr;
        socklen_t client_len = sizeof(client_addr);

        int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
        if (client_fd < 0) {
            perror("accept");
            continue;
        }

        printf("New connection from %s:%d\n",
               inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));

        /* wait on semaphore - blocks if MAX_CLIENTS are already connected */
        sem_wait(&client_sem);

        ClientSession *session = malloc(sizeof(ClientSession));
        memset(session, 0, sizeof(ClientSession));
        session->sockfd = client_fd;

        pthread_t tid;
        if (pthread_create(&tid, NULL, handle_client, session) != 0) {
            perror("pthread_create");
            close(client_fd);
            sem_post(&client_sem);
            free(session);
            continue;
        }

        pthread_detach(tid);
    }

    close(server_fd);
    close(log_pipe[1]);
    sem_destroy(&client_sem);
    return 0;
}
```

Figure 3.8: Mutex and semaphore usage for concurrency control

3.6 Data Consistency

Mutex and file locks work together to guarantee atomicity of the booking operation. The sequence — read available seats, check availability, decrement count, write updated record — executes entirely inside a pthread_mutex_lock() / pthread_mutex_unlock() block with a write file lock held throughout. This prevents race conditions, dirty reads, and lost updates even when multiple clients book seats on the same train at the same time.

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_...
========================================
Enter choice: 1

--- Available Trains ---
ID   Name                 From         To           Seats  Fare
---- ------------------ ------------ ------------ ------ -----
1    RajdhaniExp          Delhi        Mumbai       114    1500
2    ShatabdiExp          Delhi        Chandigarh   80     750
3    DurontoExp           Mumbai       Kolkata      100    1200
4    VandeExpress         Chennai      Bangalore    90     500
5    GaribRath            Delhi        Lucknow      150    350
6    GandhiExp            Delhi        Pune         200    800
END

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  0. Logout
========================================
Enter choice:
```

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_...
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  0. Logout
========================================
Enter choice: 2
Enter Train ID: 1
Enter number of seats: 2
SUCCESS: Booked! Booking ID: 5, Amount: Rs.3000
END

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  0. Logout
========================================
Enter choice:
```

Figure 3.9: Ticket booking — train list and booking confirmation

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_...
  4. Cancel Booking
  0. Logout
========================================
Enter choice: 3

--- Bookings ---
BID  User           TrainID  Seats  Amount  Status
---- -------------- -------  -----  ------  ---------
1    rahul          3        3      3600    cancelled
2    rahul          1        2      3000    cancelled
3    rahul          1        3      4500    confirmed
5    rahul          1        2      3000    confirmed
END

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  0. Logout
========================================
Enter choice:
```

Figure 3.10: Booking history displayed to the client

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_... vboxuser@ubuntu: ~/OSLab/Railway_...
========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  0. Logout
========================================
Enter choice: 4
Enter Booking ID to cancel: 3
SUCCESS: Booking cancelled and seats restored
END

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  0. Logout
========================================
Enter choice:
```

Figure 3.11: Booking cancellation confirmation

3.7 Inter-Process Communication (IPC) — Pipes

The server uses pipe() to create an unnamed pipe and fork() to spawn a child process dedicated to logging. The parent (server) process writes log messages — containing timestamps and operation details — into the write end of the pipe using write(). The child process blocks on read() from the read end and writes each message to server.log. This separates I/O-intensive logging from the main request-handling path.

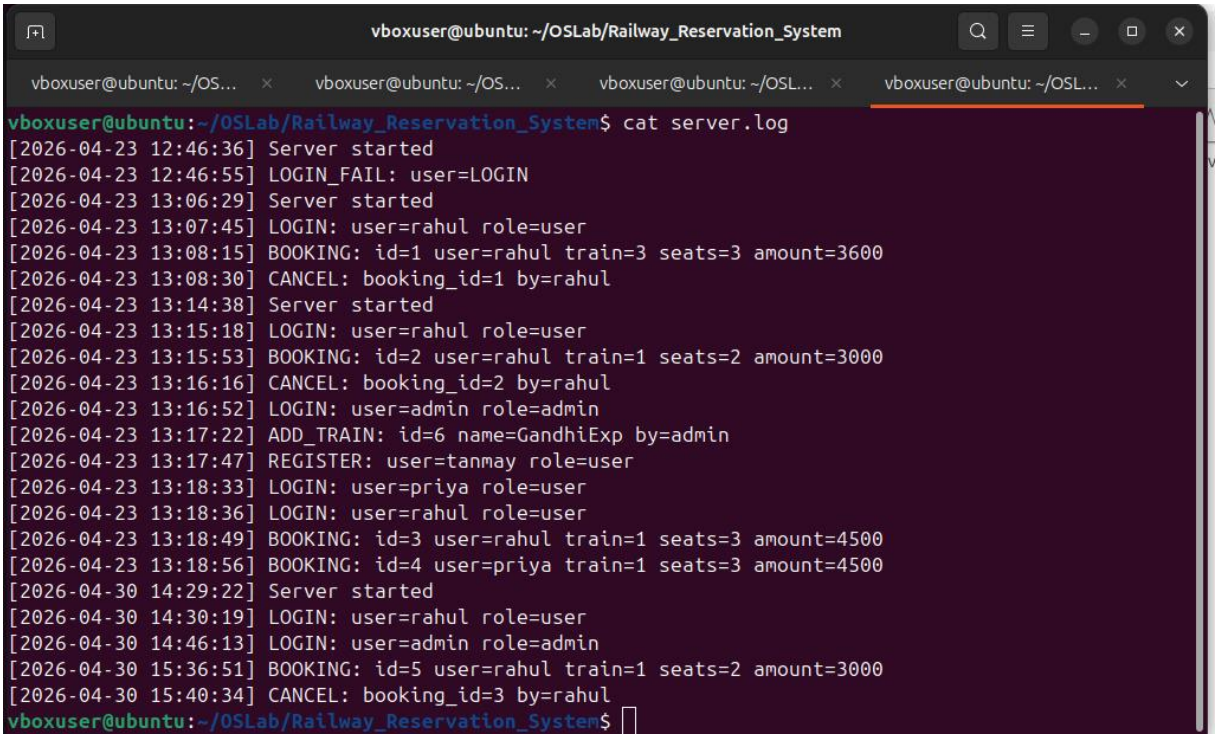

Figure 3.12: Log entries in server.log written via pipe-based IPC

```c
/* ====================================================
 * IPC - send a log message through pipe to logger process
 * ==================================================== */
void send_log(const char *msg) {
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char buf[512];
    snprintf(buf, sizeof(buf), "[%04d-%02d-%02d %02d:%02d:%02d] %s\n",
             t->tm_year + 1900, t->tm_mon + 1, t->tm_mday,
             t->tm_hour, t->tm_min, t->tm_sec, msg);
    write(log_pipe[1], buf, strlen(buf));
}


/* ====================================================
 * LOGGER CHILD PROCESS
 * Reads from the pipe and writes to server.log
 * This is the IPC demonstration - parent communicates
 * with child via pipe.
 * ==================================================== */
void logger_process(void) {
    close(log_pipe[1]); /* child only reads */

    FILE *logfp = fopen("server.log", "a");
    if (!logfp) {
        perror("open server.log");
        exit(1);
    }

    char buf[512];
    ssize_t n;

    while ((n = read(log_pipe[0], buf, sizeof(buf) - 1)) > 0) {
        buf[n] = '\0';
        fprintf(logfp, "%s", buf);
        fflush(logfp);
    }

    fclose(logfp);
    close(log_pipe[0]);
    exit(0);
}
```

```c
/* ====================================================
 * MAIN - sets up socket, pipe, logger process, accepts clients
 * ==================================================== */
int main(void) {
    struct sockaddr_in addr;
    int opt = 1;

    printf("=== Railway Reservation System Server ===\n");

    /* initialize semaphore for limiting concurrent clients */
    sem_init(&client_sem, 0, MAX_CLIENTS);

    /* initialize booking counter from existing data */
    init_booking_counter();
    printf("Booking counter initialized: %d\n", booking_counter);

    /* create pipe for IPC with logger process */
    if (pipe(log_pipe) == -1) {
        perror("pipe");
        exit(1);
    }

    /* fork logger child process */
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }

    if (pid == 0) {
        /* child process - runs the logger */
        logger_process();
        /* never returns */
    }

    /* parent continues as server */
    close(log_pipe[0]); /* parent only writes to pipe */

    signal(SIGINT, handle_sigint);
    signal(SIGCHLD, SIG_IGN); /* prevent zombie process */

    send_log("Server started");

    /* create TCP socket */
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        exit(1);
    }
```

Figure 3.13: Pipe and fork implementation for IPC-based logging

4. Admin Operations Demo

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OS...   vboxuser@ubuntu: ~/OS...   vboxuser@ubuntu: ~/OSL...   vboxuser@ubuntu: ~/OSL...
Password: admin123

Login successful! Welcome, admin (role: admin)

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  5. Add New Train (Admin)
  6. Register New User (Admin)
  0. Logout
========================================
Enter choice: 5
Train name (no spaces): Rajdhani
From station: Delhi
To station: Kanyakumari
Total seats: 95
Fare per seat: 3425
SUCCESS: Train added with ID 7
END
```

Figure 4.1: Admin successfully adds a new train

```
vboxuser@ubuntu: ~/OSLab/Railway_Reservation_System
vboxuser@ubuntu: ~/OS...   vboxuser@ubuntu: ~/OS...   vboxuser@ubuntu: ~/OSL...   vboxuser@ubuntu: ~/OSL...
========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
  4. Cancel Booking
  5. Add New Train (Admin)
  6. Register New User (Admin)
  0. Logout
========================================
Enter choice: 6
New username: Heer
Password: heer123
Role (admin/agent/user): user
SUCCESS: User registered
END

========================================
   Railway Reservation System - Menu
========================================
  1. View Available Trains
  2. Book Ticket
  3. View Bookings
```

Figure 4.2: Admin successfully registers a new user

5. Challenges Faced and Solutions

1. **Concurrent File Access**
   Multiple threads reading and writing shared files simultaneously risked data corruption. This was solved by using two-level locking: a pthread_mutex for in-memory structures and `fcntl` file locks for on-disk files, applied together before any file I/O.

2. **Preventing Deadlocks**
   Ensuring a consistent lock acquisition order (mutex first, then file lock) throughout the codebase prevented circular wait conditions that could cause the server to hang indefinitely.

3. **Handling Client Disconnection**
   An abrupt client disconnect could leave the semaphore slot permanently occupied. Cleanup code in the thread function releases the semaphore with sem_post() and closes the socket descriptor before the thread exits, regardless of how the client disconnected.

4. **Atomic Booking Operation**
   A booking involves multiple steps (read seats, check, decrement, write) that must appear atomic. Wrapping the entire sequence inside one mutex-protected section with the write file lock held ensures no interleaving is possible between threads.

6. Conclusion

This project provided practical, hands-on experience with several fundamental Operating Systems concepts that are often challenging to grasp from theory alone. Implementing **socket programming, file locking, mutexes, semaphores, and pipe-based IPC** in a single coherent application revealed how these mechanisms interact and comple-ment each other in a real-world multi-client system. The exercise of debugging **race conditions** and **ensuring data consistency** deepened the understanding of why synchro-nization primitives exist and how to apply them correctly. Overall, the Railway Reservation System served as an effective platform to bridge the gap between OS theory and systems programming practice.